BREAST CANCER DIAGNOSIS

Part 1: Data Understanding and Audit

Target Variable

The target variable is: **`diagnosis`**

- This represents whether the tumor is:
    - **M** = Malignant
    - **B** = Benign

This is the variable we are trying to predict.

Feature Variables

- `radius_mean`
- `texture_mean`
- `perimeter_mean`
- `area_mean`
- `smoothness_mean`
- `compactness_mean`
- `concavity_mean`
- `concave points_mean`
- `symmetry_mean`
- `fractal_dimension_mean`
- `radius_se`
- `texture_se`
- `perimeter_se`
- `area_se`
- `smoothness_se`
- `compactness_se`
- `concavity_se`
- `concave points_se`
- `symmetry_se`
- `fractal_dimension_se`
- `radius_worst`
- `texture_worst`
- `perimeter_worst`
- `area_worst`

- smoothness_worst
- compactness_worst
- concavity_worst
- concave points_worst
- symmetry_worst
- fractal_dimension_worst

Encode the target appropriately

here is the python script for that:

import pandas as pd

Assuming your dataset is loaded into a DataFrame called df

df = pd.read_csv("Breast Cancer Diagnosis.csv")

Encode target: M = 1, B = 0

df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

Define X and y

X = df.drop(columns=['id', 'diagnosis'])

y = df['diagnosis']

Verify Encoding

print(df['diagnosis'].value_counts())

print(df['diagnosis'].unique())

```
diagnosis
0    357
1    212
Name: count, dtype: int64
[1 0]
```

Checking for:

- **Missing values**

Total missing values per column

Count missing values per column

df.isnull().sum()

Total missing values in entire dataset

df.isnull().sum().sum()

```
np.int64(569)
```

Drop Rows with Any Missing Values

df = df.dropna()

print("Total missing values:", df.isnull().sum().sum())

```
Total missing values: 0
```

- **Duplicates**

Count duplicate rows

df.duplicated().sum()

```
np.int64(0)
```

EXPLANATION

The breast cancer diagnosis dataset contains measurements of breast tumor cell samples. Each row represents one patient case, and the columns describe different characteristics of the cell nuclei, such as radius, texture, perimeter, area, smoothness, compactness, concavity, symmetry, and fractal dimension. For most of these characteristics, three values are recorded: the mean, the standard error, and the worst (largest) value observed. The dataset also includes an ID column, which uniquely identifies each case, and a diagnosis column that indicates whether the tumor is malignant (cancerous) or benign (non-cancerous).

The main goal of this dataset is to build a model that can predict whether a tumor is malignant or benign based on the measured features. This is a binary classification problem. An accurate prediction model can assist medical professionals in making faster and more informed decisions.

Feature selection is important because many of the variables are closely related and may provide overlapping information. Removing unnecessary or highly correlated features can improve model accuracy, reduce overfitting, simplify interpretation, and make the model more efficient.

Part 2: Baseline Model Development

A- Split data into training and test sets

There are 2 steps to do it:

1- Import Required Library

Import pandas as pd

from sklearn.model_selection import train_test_split

2- Split the Data

```
df = pd.read_csv("Breast Cancer Diagnosis.csv")
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Drop the problematic 'Unnamed: 32' column instead of dropping all rows
if 'Unnamed: 32' in df.columns:
    df = df.drop(columns=['Unnamed: 32'])

X = df.drop(columns=['id', 'diagnosis'])
y = df['diagnosis']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,       # 20% for testing
    random_state=42,     # for reproducibility
    stratify=y           # keeps class distribution balanced
    )
print("Training set size:", X_train.shape)
print("Test set size:", X_test.shape)
Training set size: (455, 30)
Test set size: (114, 30)
```

Our model will learn patterns from **455 tumor cases**, each described by **30 features**.

114 cases are used to evaluate how well the model performs on unseen data.

B- Train a tree-based classifier (Decision Tree or Random Forest)

1- Decision tree

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Initialize model
dt_model = DecisionTreeClassifier(random_state=42)

# Train model
dt_model.fit(X_train, y_train)

# Make predictions
y_pred_dt = dt_model.predict(X_test)

# Evaluate
accuracy_dt = accuracy_score(y_test, y_pred_dt)
print("Decision Tree Accuracy:", accuracy_dt)
```

`Decision Tree Accuracy: 0.9298245614035088.` The model is performing very well.

2- Random forest

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Initialize model
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
# Train model
rf_model.fit(X_train, y_train)

# Make predictions
y_pred_rf = rf_model.predict(X_test)

# Evaluate
accuracy_rf = accuracy_score(y_test, y_pred_rf)
print("Random Forest Accuracy:", accuracy_rf)
```

`Random Forest Accuracy: 0.9736842105263158.` The model is performing very well.

Conclusion: Random Forest is more accurate than tree decision.

C- Evaluating performance using

1- Decision tree

```
# Predictions from Decision Tree
y_pred_dt = dt_model.predict(X_test)

# Metrics
accuracy_dt = accuracy_score(y_test, y_pred_dt)
precision_dt = precision_score(y_test, y_pred_dt)
recall_dt = recall_score(y_test, y_pred_dt)

print("Decision Tree Performance:")
print("Accuracy:", round(accuracy_dt, 4))
print("Precision:", round(precision_dt, 4))
print("Recall:", round(recall_dt, 4))
```

```
Decision Tree Performance:
Accuracy: 0.9298
Precision: 0.9048
Recall: 0.9048
```

2- Random forest

```
# Predictions from Random Forest
y_pred_rf = rf_model.predict(X_test)

# Metrics
accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf)
recall_rf = recall_score(y_test, y_pred_rf)

print("\nRandom Forest Performance:")
print("Accuracy:", round(accuracy_rf, 4))
print("Precision:", round(precision_rf, 4))
print("Recall:", round(recall_rf, 4))
```

```
Random Forest Performance:
Accuracy: 0.9737
Precision: 1.0
Recall: 0.9286
```

In summary, the Random Forest model appears to be more robust and reliable choice for breast cancer diagnosis, especially given its perfect precision in identifying malignant cases.

Part 3: Filter-Based Feature Selection

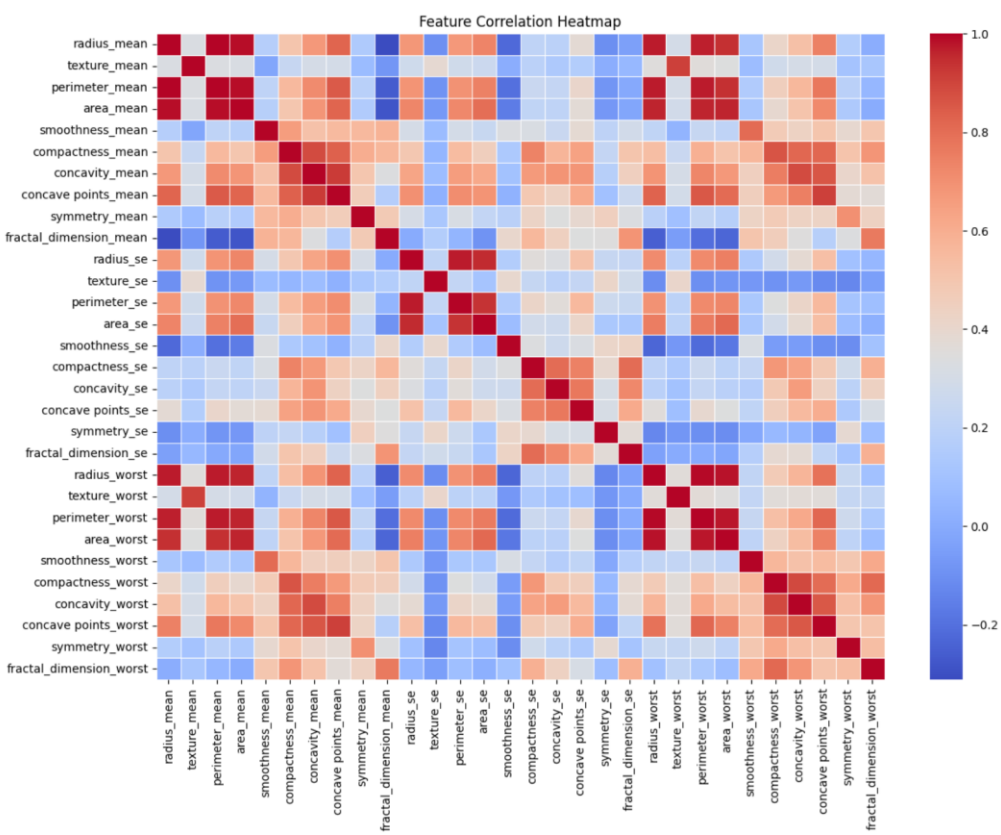
1- Perform correlation analysis on features

Step 1: Compute Correlation Matrix

```
import matplotlib.pyplot as plt
import seaborn as sns

# Exclude id and target variable
X = df.drop(columns=['id', 'diagnosis'])

# Compute correlation matrix
corr_matrix = X.corr()

# Display correlation matrix
print(corr_matrix)
```

Step 2: Visualize Correlation Heatmap

```
plt.figure(figsize=(14,10))
sns.heatmap(corr_matrix, cmap='coolwarm', linewidths=0.5)
plt.title("Feature Correlation Heatmap")
plt.show()
```



Step 3: Identify Highly Correlated Features

```
import numpy as np

# Select upper triangle of correlation matrix
upper_triangle = corr_matrix.where(
    np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))

# Find features with high correlation
high_corr = [column for column in upper_triangle.columns if
any(upper_triangle[column] > 0.9)]

print("Highly correlated features:", high_corr)
```

Highly correlated features: ['perimeter_mean', 'area_mean', 'concave points_mean', 'perimeter_se', 'area_se', 'radius_worst', 'texture_worst', 'perimeter_worst', 'area_worst', 'concave points_worst']. They are redundant and need to be dropped.

2- Define and justify a correlation threshold

In correlation analysis, a **correlation threshold** is a cutoff value used to decide when two features are considered highly correlated.

A common and reasonable threshold is: $|r|>0.9$

This means if the absolute correlation between two features is greater than 0.9, they are considered strongly correlated.

A correlation threshold of **0.9** is appropriate because it identifies only very strongly correlated features that provide nearly duplicate information. This reduces redundancy while preserving meaningful predictive variables.

3- Remove redundant features

X_reduced = X.drop(columns=to_drop)

print("Original shape:", X.shape)

print("Reduced shape:", X_reduced.shape)

```
Original shape: (569, 30)
Reduced shape: (569, 20)
```

4- Retrain the model with the reduced feature set

a- Decision tree

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.model_selection import train_test_split

# Define y from the existing df before splitting
y = df['diagnosis']

# Split the reduced feature set into training and testing data
X_train_red, X_test_red, y_train, y_test = train_test_split(
    X_reduced, y, test_size=0.2, random_state=42, stratify=y
)

dt_model_red = DecisionTreeClassifier(random_state=42)
dt_model_red.fit(X_train_red, y_train)

y_pred_dt_red = dt_model_red.predict(X_test_red)

print("Decision Tree (Reduced Features)")
print("Accuracy :", round(accuracy_score(y_test, y_pred_dt_red), 4))
print("Precision:", round(precision_score(y_test, y_pred_dt_red), 4))
print("Recall   :", round(recall_score(y_test, y_pred_dt_red), 4))
```

```
Decision Tree (Reduced Features)
Accuracy: 0.9474
Precision: 0.95
Recall: 0.9048
```

b- Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf_model_red = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model_red.fit(X_train_red, y_train)

y_pred_rf_red = rf_model_red.predict(X_test_red)

print("\nRandom Forest (Reduced Features)")
print("Accuracy :", round(accuracy_score(y_test, y_pred_rf_red), 4))
print("Precision:", round(precision_score(y_test, y_pred_rf_red), 4))
print("Recall   :", round(recall_score(y_test, y_pred_rf_red), 4))
```

```
Random Forest (Reduced Features)
Accuracy: 0.9386
Precision: 0.9487
Recall: 0.881
```

Conclusion: With the reduced feature set, the Decision Tree model is better because it achieves higher accuracy, precision, and recall compared to the Random Forest model.

Part 4: Embedded Feature Selection

For the continuation of our work, we will use the <processed_breast_cancer_data.csv> saved.

1- Train a Random Forest classifier

```
# Load dataset
df = pd.read_csv("processed_breast_cancer_data.csv")

# Drop ID column (not useful for prediction)
df = df.drop(columns=["id"])

# --- Diagnostic print statements to inspect original diagnosis values ---
print("Original unique values in 'diagnosis' column after loading CSV:")
print(df["diagnosis"].unique())
print("Value counts for original 'diagnosis' column:")
print(df["diagnosis"].value_counts(dropna=False))
# --- End diagnostic print statements ---
```

```
Original unique values in 'diagnosis' column after loading CSV:
[1 0]
Value counts for original 'diagnosis' column:
diagnosis
0    357
1    212
Name: count, dtype: int64
```

```
# Drop ID column (not useful for prediction)
df = df.drop(columns=["id"])

# The 'diagnosis' column is already numeric (0 and 1) as seen from the diagnostic prints.
# Therefore, no mapping from 'M'/'B' to 1/0 is needed.

# Drop rows where 'diagnosis' is NaN, if any exist (though unlikely after confirming its numeric nature)
df.dropna(subset=["diagnosis"], inplace=True)

X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]
```

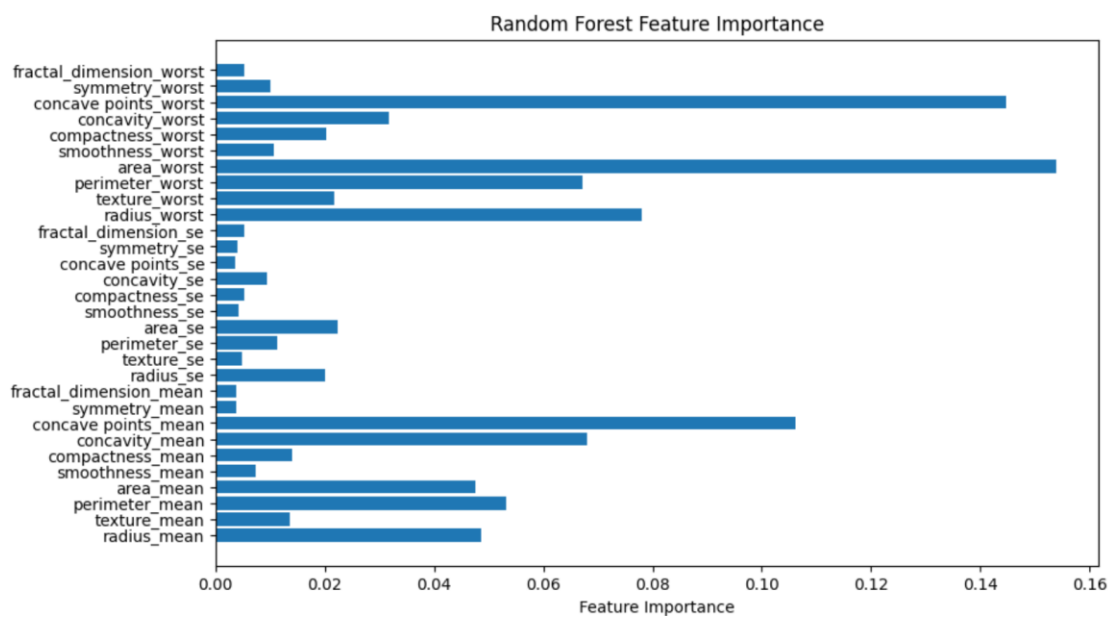
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```python
# Make predictions on the test set
y_pred = rf.predict(X_test)

# Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.9649122807017544

Classification Report:
               precision    recall  f1-score   support

           0       0.96      0.99      0.97        71
           1       0.98      0.93      0.95        43

    accuracy                           0.96       114
   macro avg       0.97      0.96      0.96       114
weighted avg       0.97      0.96      0.96       114


Confusion Matrix:
 [[70  1]
 [ 3 40]]
```

2- Extract feature importance scores

```python
import matplotlib.pyplot as plt

importances = rf.feature_importances_
features = X.columns

plt.figure(figsize=(10,6))
plt.barh(features, importances)
plt.xlabel("Feature Importance")
plt.title("Random Forest Feature Importance")
plt.show()
```



3- Rank features and select a reduced set based on an importance threshold

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv("processed_breast_cancer_data.csv")

# Drop ID column
df = df.drop(columns=["id"])

# The 'diagnosis' column is already numeric (0 and 1), so no mapping is needed here.

X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train RF
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(n_estimators=200, random_state=42)
```

```python
importances = rf.feature_importances_
feature_importance_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)

print(feature_importance_df)
```

```
                     Feature  Importance
23                area_worst    0.128549
27      concave points_worst    0.128343
22           perimeter_worst    0.127079
7        concave points_mean    0.119801
20              radius_worst    0.069273
6             concavity_mean    0.055773
0                radius_mean    0.050439
3                  area_mean    0.042542
26           concavity_worst    0.040747
2             perimeter_mean    0.038349
13                   area_se    0.023545
21             texture_worst    0.020966
25         compactness_worst    0.017708
10                 radius_se    0.016454
5           compactness_mean    0.014870
1              texture_mean    0.014745
24          smoothness_worst    0.013011
16              concavity_se    0.009897
28             symmetry_worst    0.009794
4            smoothness_mean    0.008533
12              perimeter_se    0.007931
19      fractal_dimension_se    0.006241
29  fractal_dimension_worst    0.005154
15            compactness_se    0.005008
18               symmetry_se    0.004703
9     fractal_dimension_mean    0.004487
14             smoothness_se    0.004261
11                texture_se    0.004243
17         concave points_se    0.003922
8              symmetry_mean    0.003632
```

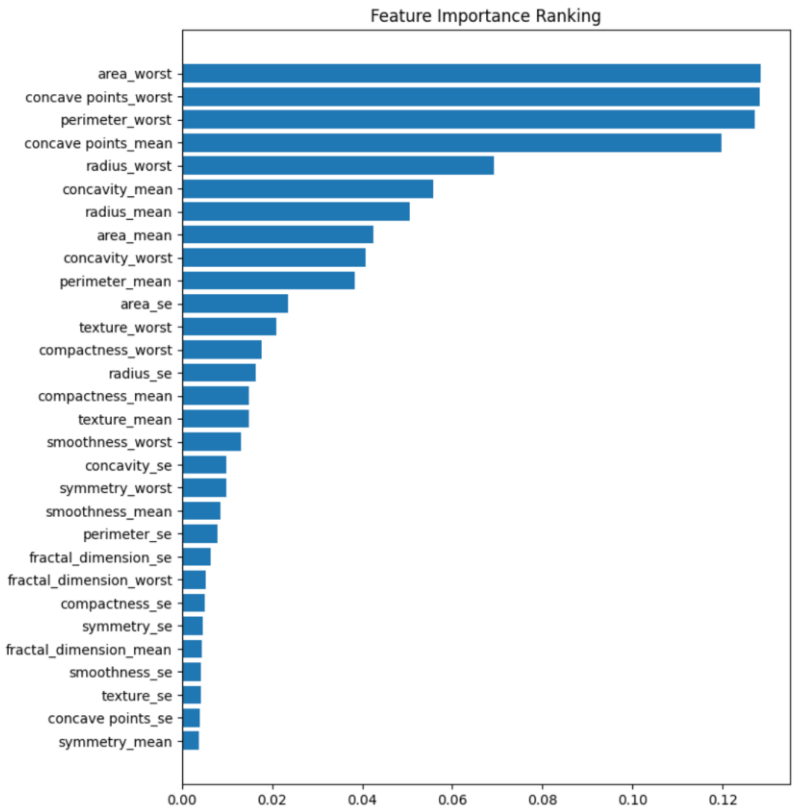
```python
selector = SelectFromModel(rf, threshold="mean", prefit=True)

X_train_reduced = selector.transform(X_train)
X_test_reduced = selector.transform(X_test)

selected_features = X.columns[selector.get_support()]
print("Selected Features:\n", selected_features)
```

```
Selected Features:
 Index(['radius_mean', 'perimeter_mean', 'area_mean', 'concavity_mean',
       'concave points_mean', 'radius_worst', 'perimeter_worst', 'area_worst',
       'concavity_worst', 'concave points_worst'],
      dtype='object')
```

```python
rf_reduced = RandomForestClassifier(n_estimators=200, random_state=42)
rf_reduced.fit(X_train_reduced, y_train)

y_pred_reduced = rf_reduced.predict(X_test_reduced)

print("Accuracy with Reduced Features:",
      accuracy_score(y_test, y_pred_reduced))
```

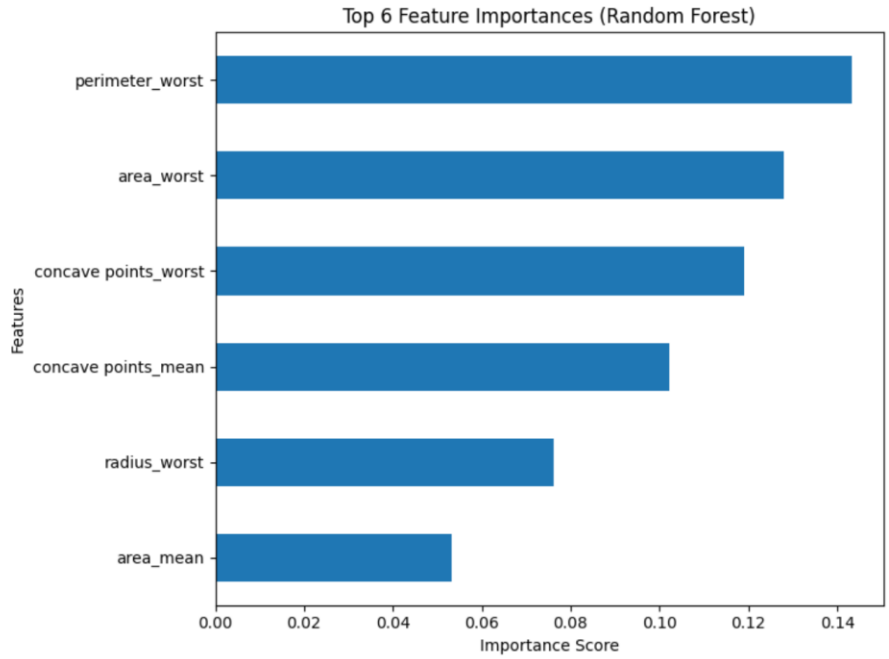
```
Accuracy with Reduced Features: 0.956140350877193
```

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(8,10))
plt.barh(feature_importance_df["Feature"],
         feature_importance_df["Importance"])
plt.gca().invert_yaxis()
plt.title("Feature Importance Ranking")
plt.show()
```




```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier

# -------------------------
# 1. Load Dataset
# -------------------------
df = pd.read_csv("processed_breast_cancer_data.csv")

df = df.drop(columns=["id"])
# The 'diagnosis' column is already numeric (0 and 1), so no mapping is needed here.
# Drop rows where 'diagnosis' is NaN, if any exist
df.dropna(subset=["diagnosis"], inplace=True)

X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]

# -------------------------
# 2. Train Random Forest
# -------------------------
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X, y)

# -------------------------
# 3. Select Top 6 Features
# -------------------------
importances = pd.Series(rf.feature_importances_, index=X.columns)
top6 = importances.sort_values(ascending=False).head(6)

print("Top 6 Features:")
print(top6)

# -------------------------
# 4. Plot Top 6 Features
# -------------------------
plt.figure(figsize=(8,6))
top6.sort_values().plot(kind="barh")
plt.title("Top 6 Feature Importances (Random Forest)")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.tight_layout()
plt.show()
```

```
Top 6 Features:
perimeter_worst         0.143298
area_worst              0.128118
concave points_worst    0.119072
concave points_mean     0.102228
radius_worst            0.076250
area_mean               0.053145
dtype: float64
```



Part 5: Wrapper Method – Recursive Feature Elimination (RFE)

1- Apply Recursive Feature Elimination (RFE) using a tree-based model

```python
# Drop ID column if exists
if "id" in df.columns:
    df = df.drop(columns=["id"])

# The 'diagnosis' column is already numeric (0 and 1), so no mapping is needed.
# Drop rows where 'diagnosis' is NaN, if any exist
df.dropna(subset=["diagnosis"], inplace=True)

X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]

# ----------------------------------------
# 2. Train-Test Split
# ----------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------------------
# 3. Define Tree-Based Model
# ----------------------------------------
rf = RandomForestClassifier(n_estimators=200, random_state=42)
```

```python
# ----------------------------------------
# 4. Apply RFE (Select Top 6 Features)
# ----------------------------------------
rfe = RFE(
    estimator=rf,
    n_features_to_select=6
)

rfe.fit(X_train, y_train);

# Selected Features
selected_features = X.columns[rfe.support_]

print("Selected Features using RFE:")
print(selected_features)

# ----------------------------------------
# 5. Reduce Dataset
# ----------------------------------------
X_train_rfe = X_train[selected_features]
X_test_rfe = X_test[selected_features]

# ----------------------------------------
# 6. Retrain Model Using Selected Features
# ----------------------------------------
rf.fit(X_train_rfe, y_train)

y_pred = rf.predict(X_test_rfe)

# ----------------------------------------
# 7. Evaluation
# ----------------------------------------
print("\nAccuracy with RFE Features:",
      accuracy_score(y_test, y_pred))

print("\nClassification Report:\n",
      classification_report(y_test, y_pred))
```

```
Selected Features using RFE:
Index(['concavity_mean', 'concave points_mean', 'radius_worst',
       'perimeter_worst', 'area_worst', 'concave points_worst'],
      dtype='object')

Accuracy with RFE Features: 0.956140350877193

Classification Report:
               precision    recall  f1-score   support

           0       0.96      0.97      0.97        71
           1       0.95      0.93      0.94        43

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```

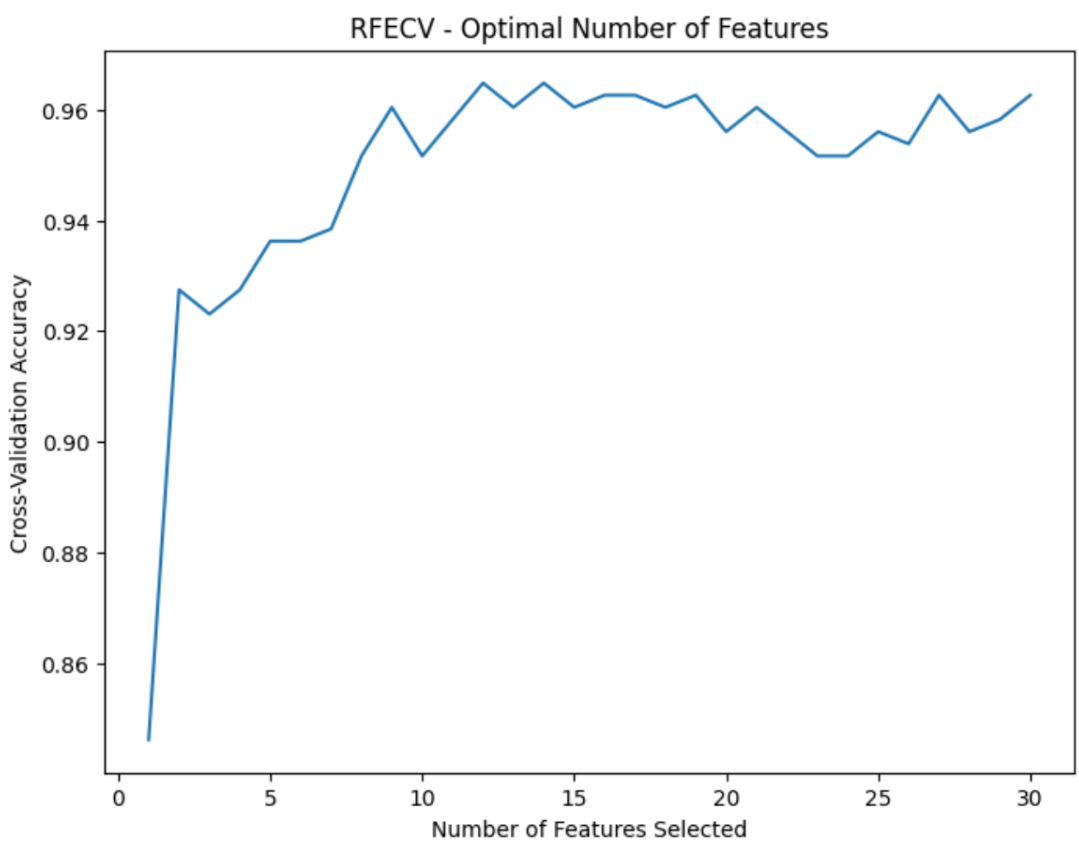
2- Identify the optimal number of features

```
df = pd.read_csv("processed_breast_cancer_data.csv")

if "id" in df.columns:
    df = df.drop(columns=["id"])

# The 'diagnosis' column is already numeric (0 and 1), so no mapping is needed.
# Drop rows where 'diagnosis' is NaN, if any exist
df.dropna(subset=["diagnosis"], inplace=True)

X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]

# ----------------------------------------
# 2. Train-Test Split
# ----------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# ----------------------------------------
# 3. Define Model
# ----------------------------------------
rf = RandomForestClassifier(n_estimators=200, random_state=42)

# ----------------------------------------
# 4. Apply RFECV
# ----------------------------------------
rfecv = RFECV(
    estimator=rf,
    step=1,
    cv=5,
    scoring="accuracy",
    n_jobs=-1)
rfecv.fit(X_train, y_train)

# ----------------------------------------
# 5. Results
# ----------------------------------------
print("Optimal number of features:", rfecv.n_features_)

selected_features = X.columns[rfecv.support_]
print("\nSelected Features:")
print(selected_features)
```

```
Optimal number of features: 12

Selected Features:
Index(['radius_mean', 'perimeter_mean', 'area_mean', 'concavity_mean',
       'concave points_mean', 'area_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'concavity_worst',
       'concave points_worst'],
      dtype='object')
```

```python
# ----------------------------------------
# 6. Evaluate on Test Data
# ----------------------------------------
X_test_optimal = X_test[selected_features]
rf.fit(X_train[selected_features], y_train)

y_pred = rf.predict(X_test_optimal)

print("\nTest Accuracy:",
      accuracy_score(y_test, y_pred))

# ----------------------------------------
# 7. Plot CV Score vs Number of Features
# ----------------------------------------
plt.figure(figsize=(8,6))
plt.plot(range(1, len(rfecv.cv_results_['mean_test_score']) + 1),
         rfecv.cv_results_['mean_test_score'])
plt.xlabel("Number of Features Selected")
plt.ylabel("Cross-Validation Accuracy")
plt.title("RFECV - Optimal Number of Features")
plt.show()
```

```
Test Accuracy: 0.956140350877193
```



3- Compare performance with, Baseline model, Filter-selected model and Embedded-selected model

```python
# ========================================
# Model Comparison:
# Baseline vs Filter vs Embedded
# ========================================

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import RFECV
from sklearn.metrics import accuracy_score
import warnings
warnings.filterwarnings("ignore")

# ----------------------------------------
# 1. Load Dataset
# ----------------------------------------
df = pd.read_csv("processed_breast_cancer_data.csv")

if "id" in df.columns:
    df = df.drop(columns=["id"])

# The 'diagnosis' column is already numeric (0 and 1), so no mapping is needed.
# Drop rows where 'diagnosis' is NaN, if any exist
df.dropna(subset=["diagnosis"], inplace=True)

X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]

# ----------------------------------------
# 2. Train-Test Split
# ----------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ========================================
# 3️⃣ BASELINE MODEL (All Features)
# ========================================
rf_baseline = RandomForestClassifier(n_estimators=200, random_state=42)
rf_baseline.fit(X_train, y_train)

y_pred_base = rf_baseline.predict(X_test)
baseline_acc = accuracy_score(y_test, y_pred_base)


    # ========================================
    # 4️⃣ FILTER METHOD (Top 6 Importance)
    # ========================================
    importances = pd.Series(
        rf_baseline.feature_importances_,
        index=X.columns
    ).sort_values(ascending=False)

    top6_features = importances.head(6).index

    rf_filter = RandomForestClassifier(n_estimators=200, random_state=42)
    rf_filter.fit(X_train[top6_features], y_train)

    y_pred_filter = rf_filter.predict(X_test[top6_features])
    filter_acc = accuracy_score(y_test, y_pred_filter)
```

```python
# ==========================================
# 5️⃣ EMBEDDED METHOD (RFECV)
# ==========================================
rf_embedded = RandomForestClassifier(n_estimators=200, random_state=42)

rfecv = RFECV(
    estimator=rf_embedded,
    step=1,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

rfecv.fit(X_train, y_train)

optimal_features = X.columns[rfecv.support_]

rf_embedded.fit(X_train[optimal_features], y_train)
y_pred_embed = rf_embedded.predict(X_test[optimal_features])
embedded_acc = accuracy_score(y_test, y_pred_embed)
```

```python
# ==========================================
# 6️⃣ Results Comparison
# ==========================================
results = pd.DataFrame({
    "Model": [
        "Baseline (All Features)",
        "Filter (Top 6 Importance)",
        "Embedded (RFECV Optimal)"],
    "Number of Features": [
        X.shape[1],
        len(top6_features),
        len(optimal_features)],
    "Accuracy": [
        baseline_acc,
        filter_acc,
        embedded_acc]})

print("\nModel Comparison Results:\n")
print(results)

print("\nOptimal Features from RFECV:")
print(list(optimal_features))
```

```
Model Comparison Results:

                       Model  Number of Features  Accuracy
0    Baseline (All Features)                  30  0.964912
1  Filter (Top 6 Importance)                   6  0.956140
2   Embedded (RFECV Optimal)                  12  0.956140
```

```python
from sklearn.metrics import classification_report
import pandas as pd

def get_metrics(y_true, y_pred):
    report = classification_report(y_true, y_pred, output_dict=True)
    return {
        "Accuracy": report["accuracy"],
        "Precision": report["weighted avg"]["precision"],
        "Recall": report["weighted avg"]["recall"],
        "F1-Score": report["weighted avg"]["f1-score"]
    }

baseline_metrics = get_metrics(y_test, y_pred_base)
filter_metrics = get_metrics(y_test, y_pred_filter)
embedded_metrics = get_metrics(y_test, y_pred_embed)

comparison_df = pd.DataFrame([
    ["Baseline", 30, *baseline_metrics.values()],
    ["Filter (Top 6)", 6, *filter_metrics.values()],
    ["Embedded (RFECV)", 12, *embedded_metrics.values()]
], columns=["Model", "Number of Features",
            "Accuracy", "Precision", "Recall", "F1-Score"])

print(comparison_df)
```

```
              Model  Number of Features  Accuracy  Precision    Recall  \
0          Baseline                  30  0.964912   0.965205  0.964912
1    Filter (Top 6)                   6  0.956140   0.956088  0.956140
2  Embedded (RFECV)                  12  0.956140   0.956088  0.956140

   F1-Score
0  0.964738
1  0.956036
2  0.956036
```

The Baseline model achieved the highest accuracy (96.49%), but the improvement over the reduced models was minimal. Both the Filter (Top 6) and Embedded (RFECV) models maintained strong performance at 95.61% accuracy. This shows that many features in the dataset are redundant and not all are necessary for high performance. Therefore, the Top 6 feature model is recommended as it provides nearly the same accuracy with much lower complexity.

Part 6: Feature Stability Analysis

1- Repeat feature selection with different random seeds or train/test splits: Stability Test: Feature Selection and Different Random Seeds

```python
df = pd.read_csv("processed_breast_cancer_data.csv")

if "id" in df.columns:
    df = df.drop(columns=["id"])

# The 'diagnosis' column is already numeric (0 and 1), so no mapping is needed.
# Drop rows where 'diagnosis' is NaN, if any exist
df.dropna(subset=["diagnosis"], inplace=True)

X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]
```

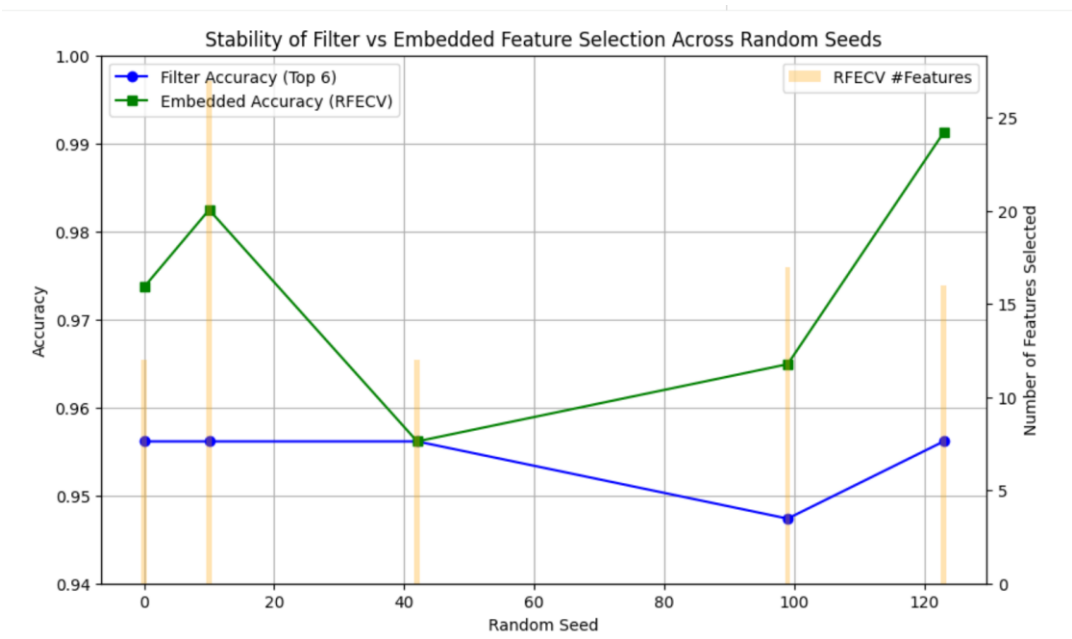
```python
# ----------------------------------------
# 2. Define Random Seeds
# ----------------------------------------
seeds = [0, 10, 42, 99, 123]

results = []

# ----------------------------------------
# 3. Repeat Experiment
# ----------------------------------------
for seed in seeds:

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=seed )

    # -------------------------
    # Filter Method (Top 6)
    # -------------------------
    rf = RandomForestClassifier(n_estimators=200, random_state=seed)
    rf.fit(X_train, y_train)

    importances = pd.Series(
        rf.feature_importances_,
        index=X.columns).sort_values(ascending=False)

    top6 = importances.head(6).index
    rf.fit(X_train[top6], y_train)
    acc_filter = accuracy_score(y_test, rf.predict(X_test[top6]))


        # ------------------------
        # Embedded Method (RFECV)
        # ------------------------
        rf_emb = RandomForestClassifier(n_estimators=200, random_state=seed)

        rfecv = RFECV(
            estimator=rf_emb,
            step=1,
            cv=5,
            scoring="accuracy",
            n_jobs=-1)

        rfecv.fit(X_train, y_train)
        selected_features = X.columns[rfecv.support_]

        rf_emb.fit(X_train[selected_features], y_train)
        acc_embedded = accuracy_score(
            y_test,
            rf_emb.predict(X_test[selected_features]))

        results.append([
            seed,
            acc_filter,
            acc_embedded,
            len(selected_features)])
```

```python
# ----------------------------------------
# 4. Results Summary
# ----------------------------------------
results_df = pd.DataFrame(
    results,
    columns=[
        "Random Seed",
        "Filter Accuracy",
        "Embedded Accuracy",
        "Embedded #Features"])

print("\nStability Results Across Different Random Seeds:\n")
print(results_df)

print("\nAverage Results:\n")
print(results_df.mean(numeric_only=True))
```

```
Stability Results Across Different Random Seeds:

   Random Seed  Filter Accuracy  Embedded Accuracy  Embedded #Features
0            0         0.956140           0.973684                  12
1           10         0.956140           0.982456                  27
2           42         0.956140           0.956140                  12
3           99         0.947368           0.964912                  17
4          123         0.956140           0.991228                  16

Average Results:

Random Seed           54.800000
Filter Accuracy        0.954386
Embedded Accuracy      0.973684
Embedded #Features    16.800000
dtype: float64
```

2- Identify features that are consistently selected and variable across runs

```python
import pandas as pd
import matplotlib.pyplot as plt

# Stability results data
data = {
    "Random Seed": [0, 10, 42, 99, 123],
    "Filter Accuracy": [0.956140, 0.956140, 0.956140, 0.947368, 0.956140],
    "Embedded Accuracy": [0.973684, 0.982456, 0.956140, 0.964912, 0.991228],
    "Embedded #Features": [12, 27, 12, 17, 16]
}

df = pd.DataFrame(data)

# Create combined figure
fig, ax1 = plt.subplots(figsize=(10,6))

# Plot accuracies
ax1.plot(df["Random Seed"], df["Filter Accuracy"], marker='o', color='blue', label="Filter Accuracy (Top 6)")
ax1.plot(df["Random Seed"], df["Embedded Accuracy"], marker='s', color='green', label="Embedded Accuracy (RFECV)")
ax1.set_xlabel("Random Seed")
ax1.set_ylabel("Accuracy")
ax1.set_ylim(0.94, 1.0)
ax1.grid(True)
ax1.legend(loc="upper left")

# Create secondary y-axis for RFECV feature count
ax2 = ax1.twinx()
ax2.bar(df["Random Seed"], df["Embedded #Features"], color='orange', alpha=0.3, label="RFECV #Features")
ax2.set_ylabel("Number of Features Selected")
ax2.legend(loc="upper right")

plt.title("Stability of Filter vs Embedded Feature Selection Across Random Seeds")
plt.show()
```



The stability analysis indicates that the filter-based feature selection method is highly consistent across different random seeds, maintaining accuracy around 95–96% with a fixed set of six features. This demonstrates strong robustness and suggests that the selected features are reliably informative regardless of how the data is split. In contrast, the embedded method (RFECV) achieved higher average accuracy (about 97%) but showed considerable variability in the number of selected features, ranging from 12 to 27. This sensitivity to training conditions may introduce uncertainty. Overall, the filter method offers greater stability and reproducibility, making it more suitable for dependable real-world deployment.

FINAL CONCLUSION

To sum up, our task was to predict breast cancer diagnosis using a dataset, focusing heavily on feature selection to find the most impactful predictors while maintaining strong model performance. We explored three main approaches: Baseline Model (All 30 Features): this served as our benchmark, achieving an accuracy of about 96.49%. then, filter Method by simply selecting the top 6 features based on their importance, we built a much simpler model that still achieved an impressive accuracy of around 95.61%. Moreover, we Embedded Method (RFECV - Optimal 12 Features) using Recursive Feature Elimination with Cross-Validation, where we identified 12 optimal features, leading to a model with an accuracy of approximately 95.61%.

Finally, we found that a highly effective and robust breast cancer prediction model can be built using a significantly reduced set of features. This not only simplifies the model but also makes it more practical for real-world application, proving the immense value of thoughtful feature selection in data analysis.